HTTP/1.1 vs HTTP/2 vs HTTP/3

how requests share the network

Watch the 3-min video

In one line: HTTP/1.1 sends one request at a time per connection; HTTP/2 multiplexes many streams over one TCP connection but still stalls on a lost packet; HTTP/3 runs on QUIC over UDP, so streams are independent, setup is faster and connections survive network changes.

A modern web page can fetch a hundred files: scripts, styles, images and API calls. How those requests share the network decides how fast the page feels. The meaning of HTTP (methods, status codes, headers) is the same in all three versions; what changes is how messages travel on the wire.

1 HTTP/1.1: one request at a time per connection

HTTP/1.1 is **text-based**. **Keep-alive** lets one TCP connection carry many requests, but only one at a time: the next request waits until the current response has fully arrived. A slow response blocks everything queued behind it. That is **head-of-line (HOL) blocking** at the HTTP level. (Pipelining tried to fix this, but responses still had to come back in order, and browsers never enabled it.)

So browsers open about **six connections per host**, each with its own TCP and TLS handshake, and still send the same bulky headers (cookies, user agent) on every request, often 500 bytes to 2 KB each.

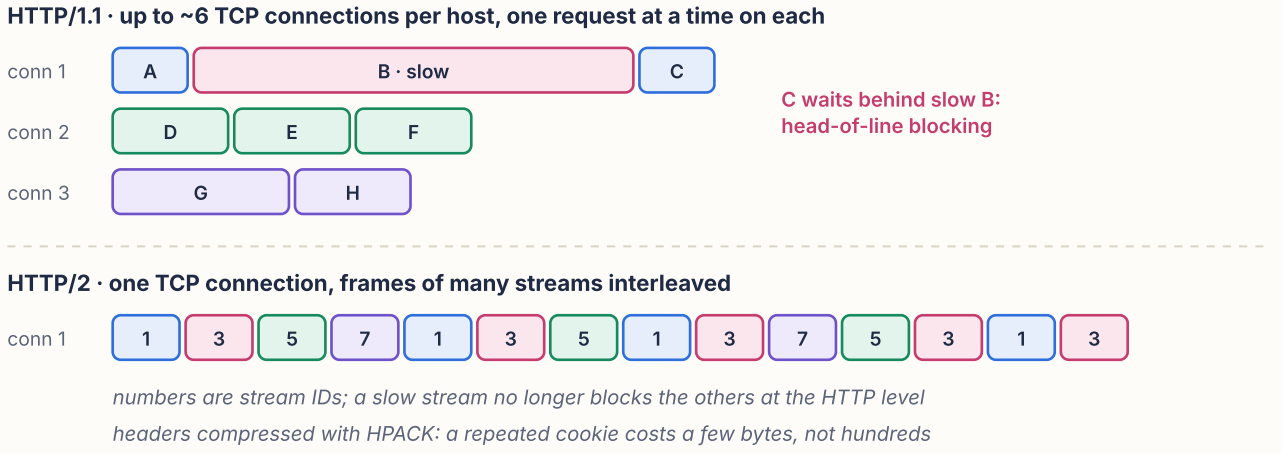


Figure 1. Top: six lanes at most, and a slow response blocks its lane. Bottom: HTTP/2 splits each request and response into frames and interleaves them on a single connection.

2 HTTP/2: binary frames and multiplexing

HTTP/2 switched to **binary framing**. Each request/response pair becomes a **stream** with an ID, cut into frames. Frames of many streams are **interleaved** on one TCP connection: that is **multiplexing**. No more waiting in line at the HTTP level, and no need for six connections (and six handshakes). Servers typically allow 100 or more concurrent streams per connection.

- **HPACK** header compression: both sides keep a table of headers already sent, so a repeated header becomes a small index of a few bytes.
- **Server push** let a server send files before they were asked for. It rarely helped and is now mostly abandoned (Chrome removed it in 2022).

- Browsers only use HTTP/2 over TLS; the version is agreed in the TLS handshake via **ALPN** (`h2`).

gRPC is built on HTTP/2. Every call is a stream, so one connection can carry thousands of concurrent calls, including long-lived **streaming calls in both directions**.

3 The catch: TCP head-of-line blocking

HTTP/2 still runs on **TCP**, and TCP delivers bytes strictly in order. If one packet is lost, every byte after it waits in the receiver's kernel buffer until the retransmission arrives, even bytes that belong to other streams. Lose a single packet and **every stream on that connection stalls**. On a lossy mobile network, one HTTP/2 connection can even be slower than six HTTP/1.1 connections, where a loss only stalls one of the six.

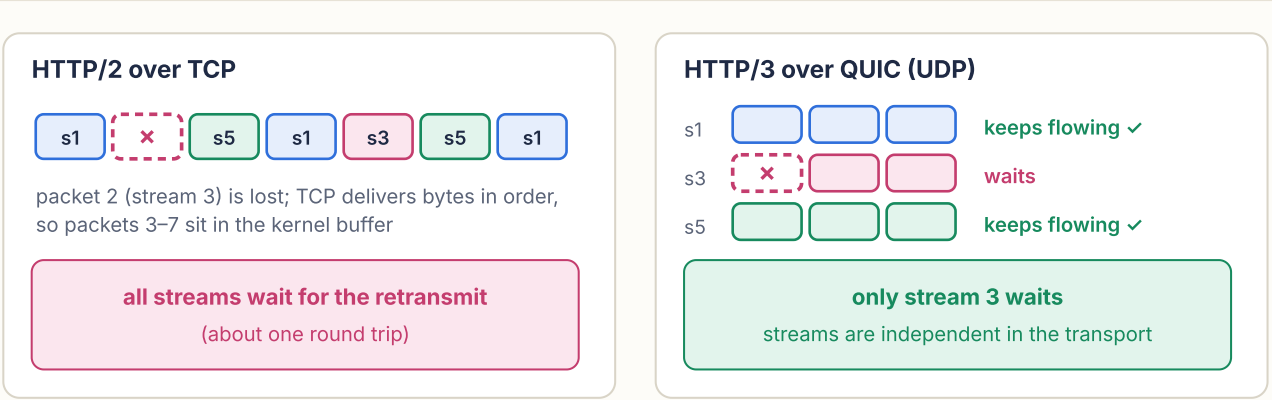


Figure 2. HTTP/2 moved head-of-line blocking from HTTP down to TCP. QUIC tracks ordering per stream, so a lost packet only delays the stream it belongs to.

4 HTTP/3: QUIC over UDP

HTTP/3 runs on **QUIC**, a transport protocol built on top of UDP. It keeps HTTP/2's ideas (streams, header compression, now called QPACK) and fixes the transport:

- **Independent streams.** QUIC handles ordering and retransmission per stream, so a lost packet only stalls its own stream.
- **Faster setup.** TLS 1.3 is built into QUIC, so the transport and encryption handshakes happen together in **one round trip**, or **zero** on resumption (0-RTT, idempotent requests only).
- **Connection migration.** A connection is identified by a **connection ID**, not by IP address and port. When your phone moves from Wi-Fi to 4G, the connection migrates instead of starting over.

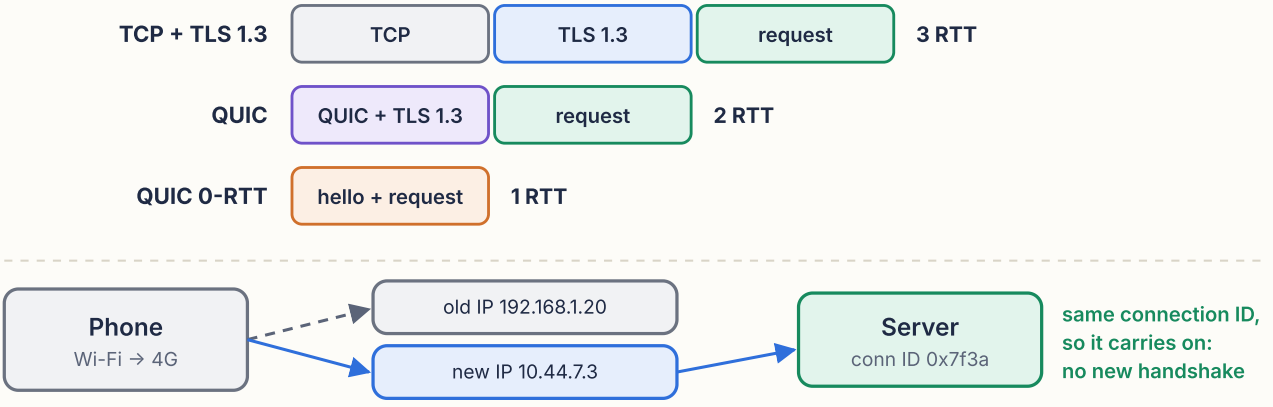


Figure 3. Top: round trips before the first response byte. QUIC merges the TCP and TLS handshakes. Bottom: the client's IP changes, but the connection ID doesn't, so the server keeps the same connection.

Why build on UDP instead of fixing TCP? TCP lives inside operating-system kernels, and countless routers, firewalls and middleboxes expect it to behave in a fixed way (it has **ossified**). Changing it would take decades. UDP passes through almost everywhere, and QUIC runs in **user space**, inside the browser or server, so it evolves as fast as they update. Some networks still block UDP, so browsers learn about HTTP/3 from an `Alt-Svc` header (or DNS) and fall back to HTTP/2 over TCP if QUIC fails.

	HTTP/1.1	HTTP/2	HTTP/3
Format	Text	Binary frames	Binary frames
Transport	TCP	TCP	QUIC over UDP
Requests per connection	One at a time	Many streams, multiplexed	Many independent streams
Head-of-line blocking	HTTP level	TCP level (packet loss)	Only within one stream
Headers	Repeated in full	HPACK	QPACK
Setup (with TLS 1.3)	2 RTT	2 RTT	1 RTT, or 0 on resume
Network change	Reconnect	Reconnect	Migrates (connection ID)

WORKED EXAMPLE

A page loads 60 files over a 100 ms round-trip mobile link with 2% packet loss. HTTP/1.1: 6 connections × (TCP + TLS) handshakes, and 60 requests queued about 10 deep per lane, so the page needs at least ~12 round trips (≈ 1.2 s) even with fast servers. HTTP/2: one handshake, all 60 requests sent at once, so far fewer round trips; but with 2% loss, each lost packet freezes all 60 streams for about a round trip. HTTP/3: one combined handshake (100 ms less), and a loss only delays the one file it belongs to.

COMMON MISTAKES

- Saying HTTP/2 removed head-of-line blocking completely: it moved it to TCP.
- Keeping HTTP/1.1 tricks on HTTP/2 (domain sharding, sprite sheets, giant bundles): extra domains mean extra connections and lose multiplexing.
- Assuming HTTP/3 always works: corporate firewalls may block UDP 443; keep the HTTP/2 fallback.
- Thinking a new version changes your API: methods, status codes and headers mean the same thing.

5 Key terms

Keep-alive: reuse one TCP connection for many requests

Multiplexing: interleaving frames of many streams

QUIC: UDP-based transport with built-in TLS 1.3

HOL blocking: one slow or lost item delays everything behind it

HPACK / QPACK: header compression for HTTP/2 / HTTP/3

Connection ID: identifies a QUIC connection, not IP + port

Stream: one request/response inside a connection

ALPN: TLS extension that picks h2 or http/1.1

Ossification: middleboxes freezing a protocol's behaviour

6 Interview questions

Q1. What problem does HTTP/2 solve? BASIC

HTTP/1.1 handles one request at a time per connection, so browsers open ~6 connections and a slow response blocks its lane. HTTP/2 multiplexes many streams over one connection and compresses headers with HPACK.

Q2. Why was HTTP/3 built on UDP? BASIC

To escape TCP's head-of-line blocking and its ossified, kernel-bound design. QUIC on UDP runs in user space, so it can offer independent streams, a combined 1-RTT handshake with TLS 1.3, and connection migration, and still evolve quickly.

Q3. What is head-of-line blocking? BASIC

When one slow or lost item delays everything queued behind it. In HTTP/1.1 it's a slow response on a connection; in HTTP/2 it's a lost TCP packet stalling all streams; HTTP/3 limits it to one stream.

Q4. Your mobile users on flaky networks see slow loads even after moving to HTTP/2. Why, and what helps? SCENARIO

Packet loss: every lost TCP packet stalls all streams on the single connection. Enable HTTP/3 at the CDN or load balancer (with HTTP/2 fallback); QUIC limits stalls to one stream and survives Wi-Fi/4G switches without reconnecting.

Q5. Why does gRPC use HTTP/2? SCENARIO

Streams: one connection carries thousands of concurrent calls, binary framing suits protobuf, and long-lived streams allow client, server and bidirectional streaming calls. It also gets HPACK and per-stream flow control.

Q6. You enabled HTTP/3 but some office users still use HTTP/2. Is something broken? SCENARIO

Probably not. Their firewall likely blocks UDP 443, so the browser falls back to TCP. Browsers discover HTTP/3 via Alt-Svc or DNS HTTPS records and use it only when QUIC actually connects.

Q7. Can HTTP/2 be slower than HTTP/1.1? TRICKY

Yes, on lossy links. With six HTTP/1.1 connections, a lost packet stalls one of them; with one HTTP/2 connection, it stalls every stream, and one connection also gets a single share of TCP congestion control.

Q8. Why can't we just fix head-of-line blocking in TCP? TRICKY

TCP is implemented in OS kernels and inspected by routers, firewalls and NATs that drop anything unexpected. Deploying a change takes many years. QUIC encrypts nearly all its headers, so middleboxes can't freeze its behaviour again.

Next: SD-B-014 · WebSockets backendroadmap.com